

UMA and Across Incremental Audit

OpenZeppelin Security - March 2, 2026

Summary

Type: Cross Chain

Timeline: From 2026-01-19 → To 2026-01-28

Languages: Solidity

Findings

Total issues: 13 (10 resolved, 1 partially resolved)

Critical: 0 (0 resolved) · High: 0 (0 resolved) · Medium: 2 (1 resolved) · Low: 5 (4 resolved, 1 partially resolved)

Notes & Additional Information

5 notes raised (4 resolved)

Client Reported Issues

1 issue reported (1 resolved)

This report summarizes the result from diff audits performed on three separate scopes across three repositories. Below each scope is listed separately.

This site uses cookies to offer you a better user experience. Information about the cookies we use can be found in our Privacy Policy under "Cookie Policy". You can reject all non-essential cookies by choosing "Decline" or set your own preference under "Cookies settings".

[Cookies settings](#)

Accept

Decline

and HEAD commit at [fcc1869](#).

We also reviewed [PR #5](#) at commit [87cd8b2](#).

In scope were the following files:

```
src/common/implementation/LockableUpgradeable.sol
src/optimistic-oracle-v2/implementation/ManagedOptimisticOracleV2.sol
src/optimistic-oracle-v2/implementation/OptimisticOracleV2.sol
src/optimistic-oracle-v2/interfaces/ManagedOptimisticOracleV2Interface.sol
src/optimistic-oracle-v2/interfaces/OptimisticOracleV2Interface.sol
```

Scope 1 Changeset Overview

The `ManagedOptimisticOracleV2` contract adds resolver and resolver admin roles with a new initializer, restricting settlement to resolvers only and making `settleAndGetPrice` require prior settlement. It replaces `minimumLiveness` with a bounded `minimumDisputeWindow`, adds `setDefaultLiveness`, and decouples default liveness from the minimum dispute window. Dispute flow is updated so that expired requests are treated as proposed until a resolver settles, and liveness validation uses `minimumDisputeWindow` as a lower bound.

The `OptimisticOracleV2` parent contract now records `proposalTime`, exposes a `_getStateForDispute` hook, makes `_settle` overridable, and introduces `LEGACY_DEFAULT_LIVENESS` for legacy event-based request timestamps. The interfaces, events, and errors have been updated accordingly.

In `LockableUpgradeable`, the `nonReentrant` modifier now calls a combined `_preEntranceCheckAndSet()` helper but otherwise keeps the same guard behavior.

Scope 1 Trust Assumptions

Throughout the scope 1 codebase, the following trust assumptions were identified:

- Since settlement is permissioned in `ManagedOptimisticOracleV2` and only addresses with `RESOLVER_ROLE` can call `settle`, resolvers will resolve in a timely and correct manner. It is also trusted that the `RESOLVER_ADMIN_ROLE` (self-administered) will curate resolvers safely and responsibly via `grantRole` / `revokeRole`. If `RESOLVER_ADMIN_ROLE` is compromised, a contract upgrade is needed by the `UpgradeAdmin`.
- The `ConfigAdmin` can set both `defaultLiveness` and `minimumDisputeWindow`, with `minimumDisputeWindow` bounded between 5 minutes and `defaultLiveness`, and `defaultLiveness` still bounded by the parent contract's max liveness. This allows the config admin to control the earliest resolution time and global liveness behavior.
- `UpgradeAdmin` can call `initializeV2` and will properly perform the initialization. Deployments with `defaultLiveness` < 5 minutes must increase it before calling `initializeV2` to avoid reverts.

Scope 2

Scope 2 comprises the [UMAprotocol/protocol-private](#) repository, with BASE commit at [82936c4](#) and HEAD commit at [14b5140](#).

In scope was the following file:

```
packages/core/dispute-router/src/DisputeRouter.sol
```

Scope 2 Changeset Overview

The diff introduced a new `dispute-router` module inside the UMA core package. The main `DisputeRouter.sol` contract is an owner-gated UUPS upgradeable contract implementing a council-first resolution with forward delay to oracle via immutable UMA finder.

Registered contracts can `requestPrice` which is first resolved locally by a trusted council address or escalated by the council at any time to the oracle. When the forward delay is passed, anyone can forward a price request to an oracle and uses. `hasPrice` and `getPrice` can be used to retrieve the result from either local resolution is the price was resolved by the council, or by remote oracle resolution if the price request was forwarded.

Scope 2 Trust Assumptions

Throughout the scope 2 codebase, the following trust assumptions were identified:

- The `Owner` is honest when performing upgrades and admin changes, and the council is honest and not compromised so it can resolve prices unilaterally correctly or instantly escalate, as its resolutions are final and cannot be overridden by the oracle. Thus, incorrect resolutions from the council are not reversible unless via new functionalities introduced from an upgrade.
- The oracle is truthful and live. Once forwarded, correctness/availability depends entirely on the underlying oracle's price retrieval functions. In other words, it is expected that any forwarded requests will be resolved eventually, and the resolved price will be available via the `DisputeRouter`. If the oracle address is updated, any resolved prices that were previously available through the dispute router should remain so, and it is trusted that the new oracle will implement fallback logic to correctly retrieve any prices that were resolved by the old oracle.
- The `Finder` immutable address is pointed at a trustworthy registry as all registration gating with `onlyRegisteredContract` relies on accurate registry state.

Scope 3

Scope 3 comprises the [across-protocol/contracts](#) repository, with BASE commit at [c4dd51e](#) and HEAD commit at [d3bc16e](#).

In scope were the following files:

```
contracts/Lens_SpokePool.sol
contracts/ZkSync_SpokePool.sol
contracts/handlers/HyperliquidDepositHandler.sol
contracts/SpokePoolPeriphery.sol
```

Scope 3 Changeset Overview

The `Lens_SpokePool` constructor now takes an `_l1ChainId` value and forwards it. It inherits the `ZkSync_SpokePool` contract that switches ERC-20 withdrawals from the deprecated `zkERC20bridge` to ZkSync Gateway's `L2AssetRouter`. The diff adds constants for related ZkSync specific addresses, stores immutable `l1ChainId`, computes `assetId` and encodes burn data for withdrawals accordingly.

The `SpokePoolPeriphery` contract updates the `depositNative` function signature by adding a `depositor` argument (used instead of using `msg.sender`) and the interface is also updated accordingly.

The `HyperliquidDepositHandler` introduces `AccountActivationMode` with three options, either `none` or `fromUserFunds` or `fromDonationBox`. Only when an account uses fund from the `DonationBox` for account activation, a signature from the contract `signer` is required. Depositors can now choose whether to use a signature to sponsor account creation or not. This means the signature will not need to be transmitted outside of account creation use cases. The changes also allow depositing funds into a custom DEX on Hyperliquid, not just spot.

Scope 3 Trust Assumptions

Throughout the scope 3 codebase, the following trust assumptions were identified:

- The `Lens_SpokePool` is instantiated with the correct `_l1ChainId`.
- In the `ZkSync_SpokePool` contract, all hardcoded constants are correct.
- In the `SpokePoolPeriphery` contract, the caller-supplied `depositor` value is a correct non-zero address otherwise one may not receive refunds.
- In the `HyperliquidDepositHandler` contract, the designated signer is not compromised.

Medium Severity

Invalid `destinationDex` Can Strand Non-USDC Bridged Funds on Hypercore

`HyperliquidDepositHandler` decodes an arbitrary `destinationDex` from user- or relayer-provided calldata and forwards it to Hypercore without validation or whitelisting. When a deposit encodes an invalid DEX, the handler will still bridge funds and attempt to route to that DEX, despite the destination being invalid. This can strand funds on the corresponding handler in Hypercore and would require

manual rescue.

The issue arises because `destinationDex` is never checked against known/active DEX identifiers and `transferERC20EVMToCore` blindly issues a `SEND_ASSET_TO_DEX` action whenever the value differs from `CORE_SPOT_DEX_ID`. For USDC, the `destinationDex` is further **checked** against its enabled list.

Consider constraining `destinationDex` to a vetted set or providing limited support to custom DEXs for non-USDC token deposits.

Update: Acknowledged, will resolve. The team stated:

The `destinationDex` will be picked by our API and the user is free to use it as is or modify it. If the user modifies the `destinationDex` such that it strands the funds then that is on the user.

Account Activation DoS via Dust Deposit in `FromDonationBox` Mode

The `HyperliquidDepositHandler._depositToHypercore` function incorrectly handles small user deposits when `mode == FromDonationBox` and tokens have positive `decimalDiff` (i.e., Core has more decimals than EVM). When a user provides a tiny `evmAmount`, the donation box adds `activationFeeEvm` to form `totalEvmAmount`, but during the **transfer**, `maximumEVMSendAmountToAmounts` rounds down during decimal conversion. If the resulting `_amountCoreToReceive` equals `accountActivationFeeCore`, the **condition** in `transferERC20EVMToCore` fails (`_amountCoreToReceive > accountActivationFeeCore` is `false`), preventing any Core transfer. Consequently, the user's Hypercore account remains unactivated and receives zero tokens, yet `accountsActivated[user] = true` is **already set**, permanently blocking future sponsored activations for that address and wasting donation box funds. This issue does not affect `FromUserFunds` mode, which validates `depositCore > accountActivationFeeCore`.

Consider adding a similar validation in the `FromDonationBox` mode to ensure that `_amountCoreToReceive > accountActivationFeeCore` after combining the user's deposit with the sponsored fee. Alternatively, consider reverting the transaction before marking the account as activated.

Update: Resolved in [pull request #1268](#). The team stated:

We believe this is no longer an issue after the fixes to the latest audit got merged. Please see PR 1268.

Low Severity

`MultiCaller.multicall` **Corrupts Revert Data by Decoding Failures as**
`Error(string)`

`ManagedOptimisticOracleV2` inherits `MultiCaller` to support atomic batching via `multicall(bytes[])`. The implementation delegates to `address(this).delegatecall(...)` and attempts to bubble failures from inner calls (see `MultiCaller.multicall`).

However, the failure handling in `multicall` assumes that the returned `result` is always `Error(string)`-encoded. If `result.length < 68`, it executes `revert()` and discards the original revert data entirely. If `result.length >= 68`, it strips 4 bytes and `abi.decode`s a `string`, which can revert or produce nonsense for custom errors and panic codes. Since `ManagedOptimisticOracleV2` uses 0-argument custom errors such as `RequesterNotWhitelisted()` (raised in `requestPrice`), multicall failures can become opaque and harder to integrate with `try/catch` decoding and offchain tooling.

Consider bubbling raw revert bytes from the inner `delegatecall` (for example, using assembly `revert(add(result, 0x20), mload(result))`) instead of attempting to decode `Error(string)`.

Update: Resolved in [pull request #4](#) at commit [e44c594](#). The team stated:

We address the issue by bubbling raw revert bytes from the inner `delegatecall`.

First Proposer Can Freeze Default Request Settings by Front-Running Post-Request Configuration

`OptimisticOracleV2.requestPrice` creates a request with default `RequestSettings`. The requester can adjust request parameters using setters such as `setBond`, `setCustomLiveness`, `setRefundOnDispute`, `setEventBased`, and `setCallbacks`. Each requires the state be `State.Requested`.

However, `proposePriceFor` is permissionless and sets `request.proposer` and `request.expirationTime`, causing `_getState` to stop returning `State.Requested`. Any pending or subsequent configuration call then reverts, allowing a proposer to lock in default liveness and bond settings and to prevent intended refund/callback behavior. This reduces the requester-controlled dispute window and can weaken the economic security assumptions for that request.

Consider offering an atomic request-and-configure entrypoint (e.g., `requestPrice` that accepts `RequestSettings`, or a multicall-based helper) so that configuration cannot be front-run. Consider adding proposer-side expected settings parameters (e.g., maximum liveness and minimum bond) that revert if the effective settings differ at execution time.

Update: Partially Resolved in [pull request #8](#) at commit [bf1909f](#). The team stated:

The current design has operated safely in production for around 5 years. The system provides multiple patterns for front-run resistant configuration:

1. Atomic contract calls (used by all production integrators)
2. Inherited `multicall()` function
3. Request Manager pre-configuration

We have updated NatSpec documentation to clarify the intended behavior:

- Added notes to `requestPrice()` explaining that settings can be modified until proposal and should be configured atomically
- Added dev notes to all setter functions (`setBond()`, `setCustomLiveness()`, `setRefundOnDispute()`, `setEventBased()`, `setCallbacks()`) clarifying they require `State.Requested` and should be called atomically to prevent front-running
- Added notes to `proposePrice()` and `proposePriceFor()` explaining they lock in settings by transitioning to `State.Proposed`
- Enhanced documentation for Request Manager functions (`requestManagerSetBond()`, `requestManagerSetCustomLiveness()`) emphasizing pre-configuration capability
- Clarified that `getManagedRequestId()` omits timestamp to enable front-run resistant pre-configuration

Missing Bridge-Liquidity Guard Can Lock HyperEVM Deposits in the Asset Bridge Address

`HyperliquidDepositHandler._depositToHypercore` does not check whether the destination-side bridge has sufficient liquidity before sending funds to the asset bridge address.

`HyperCoreFlowExecutor` explicitly treats this as a funds-loss scenario and gates bridging via `isCoreAmountSafeToBridge`, which checks the Core-side `spotBalance` of `toAssetBridgeAddress`. If the bridge is depleted, the EVM-side ERC20 transfer can still succeed while the Core-side credit does not, leaving funds at `toAssetBridgeAddress(tokenId)` with no on-chain recovery path for either the end user or the handler (the handler can only sweep its own balances, e.g., `sweepCoreFundsToUser`).

Consider computing the implied Core amount (via `HyperCoreLib.maximumEVMSendAmountToAmounts`) and enforcing a pre-bridge liquidity check with `isCoreAmountSafeToBridge`, reverting or falling back (e.g., returning funds on HyperEVM) when bridging is unsafe. Moreover, consider aligning the handler's bridge behavior with the `HyperCoreFlowExecutor` guard-and-fallback pattern to avoid sending user funds to the asset bridge address without safety measures.

Update: Resolved in [pull request #1273](#).

Account Activation Sponsorship Signatures Lack Domain Separation and Expiry

`_verifySignature` [authorizes](#) donation-box-funded activation using `ECDSA.recover(keccak256(abi.encode(user)), signature)` without domain separation or an expiry. The signed message omits contract-specific data (e.g., contract address, chain ID) and time-bounding metadata, so the signature is a raw hash of `user` that can be replayed, in theory, wherever an identical hash is accepted. Lack of nonce or deadline further allows indefinite reuse.

In this current specific case of account activation sponsorship authorization, the scope is limited to one-time use and no other immediate cross-context replay is possible. Nonetheless, it is advised to future-proof signature authorization and adopt industry best practices.

Consider adopting EIP-712-style domain separation (including contract address, chain ID, and intent identifier), expanding the signed payload to cover the action being authorized, and adding an explicit expiry to limit signature lifetime.

Update: Resolved in [pull request #1300](#).

Missing `uint64` Bound Checks Can Truncate Core Amounts Derived from Large `uint256` Deposits

The HyperEVM-to-HyperCore conversion logic in `maximumEVMSendAmountToAmounts` returns a `uint64 amountCoreToReceive`, while callers (including `HyperliquidDepositHandler._depositToHypercore`) accept `uint256` EVM amounts and forward them into `transferERC20EVMTToCore`.

`maximumEVMSendAmountToAmounts` performs unchecked narrowing casts (e.g., `amountCoreToReceive = uint64(amountEVMToSend)` and `uint64(amountEVMToSend * scale)`) while only documenting, but not enforcing, that the result is within `uint64` range. If `maximumEVMSendAmount` is large enough for a given `decimalDiff`, these casts will truncate rather than revert, potentially causing a fill to debit a large `uint256` amount on HyperEVM while only transferring a truncated `uint64` amount on Hypercore.

Consider adding explicit bounds checks before narrowing to `uint64` (e.g., reverting when the computed Core amount exceeds `type(uint64).max`) and propagating an upper-bound check to user-facing entrypoints (such as `depositToHypercore` / `handleV3AcrossMessage`). In addition, consider documenting any protocol-level or off-chain constraints that are relied upon to keep converted Core amounts within `uint64` range.

Update: Resolved in [pull request #1299](#).

Notes & Additional Information

Misleading Documentation

Throughout the codebase, multiple instances of misleading documentation were identified:

- The [documentation](#) for `requestManagerSetCustomLiveness` states that it will "override any subsequent calls to `setLiveness` by the requester". However, this function does not exist, and the documentation should instead refer to `setMinimumDisputeWindow`.
- The `setOracle` [documentation](#) states that "forwarded but unresolved requests will be orphaned". However, the actual implementation orphans ALL forwarded requests regardless of resolution status if the new oracle is not correctly configured with a fallback mechanism. As such, consider being explicit about the intended functionality that the new oracle is expected to implement fallback logic to the previous oracle to preserve access to resolved prices.

- The [documentation](#) for `refund` in the `priceDisputed` callback function states that it is the received refund. However, it can be [deferred](#) instead if transfer fails, which should be explicitly mentioned.

Consider addressing any instances of misleading documentation to improve the overall clarity and maintainability of the codebase.

Update: Resolved in [pull request #6](#) at commit [84668db](#) as well as [pull request #10](#) at commit [72f4968](#).

`_verifySignature` Declares to Return `bool` But Does Not Have Explicit Returns

The `_verifySignature` function of the `HyperliquidDepositHandler` contract declares that it returns a boolean value based on the correctness of the provided signature. However, it does not contain explicit return statements and instead relies on this function reverting when a signature is incorrect.

Consider removing the return type and treating `_verifySignature` as a revert-only guard. Alternatively, consider explicitly returning `true` on success and updating call sites to consume the return value consistently.

Update: Resolved in [pull request #1273](#).

Lack of Zero-Depositor Check

`SpokePoolPeriphery.depositNative` forwards the provided depositor directly to the spoke without validation, and all other deposit entry points funnel through the same path, allowing `address(0)` as depositor. The core `_depositV3` function only checks that the value fits inside 160 bits, so the zero address passes through unchanged.

Consider rejecting `address(0)` as depositor at the periphery entry points and reinforcing the check in `_depositV3`, reverting early when a zero depositor is provided.

Update: Acknowledged, not resolved. The team stated:

Periphery contract relies on the TX sender to provide a correct `depositor` in all of the `external` ly callable functions, `depositNative` follows this pattern. Also, if we protect against setting the zero address, the integrator can still just specify an incorrect address, leading to the loss of funds just as well.

Typographical Error

In the `ManagedOptimisticOracleV2` contract, [self-govening](#) should be `self-governing`.

Consider fixing the aforementioned typographical error to improve the clarity of the codebase.

Update: Resolved in [pull request #7](#) at commit [b821e23](#).

Potential Early Revert With Unsupported L2 Assets

`ZkSync_SpokePool._bridgeTokensToHubPool` always derives an `assetId` and calls the asset router even when the L2 token has no L1 mapping. For instance, a user deposits an Era-native token that is not registered in the router. `_getAssetId` hashes `address(0)` and the subsequent `l2AssetRouter.withdraw` call reverts due to an unrecognized `assetId`.

Consider adding an explicit guard in `_getAssetId` to revert with a clear error when `l1TokenAddress == address(0)`, avoiding the router call and clarifying unsupported assets in the docstring.

Update: Resolved in [pull request #1280](#).

Client Reported

Account Activation Gas Token May Be Different From User-Specified Transfer Token

The current implementation assumes that the activation fee can be paid by withholding part of the transferred asset. However, only certain eligible activation gas tokens can be used. If the handler is configured to accept a token that is not activation-eligible, the EVM-side transfer (and optional `DonationBox` withdrawal) can still happen, while the Core action can fail to activate/credit the user, leaving funds stuck under the handler's Hypercore account. This is compounded because `accountsActivated[user]` is set before bridging (see `accountsActivated[user] = true`), so subsequent attempts can revert with `AccountAlreadyActivated()` even when `coreUserExists(user)` remains `false`.

Consider enforcing that `FromUserFunds` / `FromDonationBox` are only usable when the deposited token is activation-eligible and paying the activation fee via an explicit transfer in a known activation token instead of withholding the transferred asset. Also, consider setting `accountsActivated[user]` only after activation is confirmed, or adding a bounded retry/reset mechanism when activation does not complete.

Update: Resolved in [pull request #1273](#).

Conclusion

The review covered three scopes:

- **Early Resolver:** `ManagedOptimisticOracleV2` now treats default liveness as a minimum and extends disputes until a permissioned resolver settles, with new resolver admin roles, request metadata, and stricter settle behavior.
- **Dispute Router:** Adds a UUPS upgradeable oracle wrapper that lets a council resolve disputes locally, delay/forward to an underlying oracle, or immediately escalate, with a passthrough option.
- **Across:** Migrates Lens and ZkSync to the new ZkGateway, expands Hyperliquid deposits (optional sponsorship signature and custom DEX), and lets `depositNative` set a depositor argument instead of always `msg.sender`.

These changes represent iterative improvements to an already well-structured codebase with good documentation. Two medium-level issues were discovered in the `HyperliquidDepositHandler` contract of the Across scope, along with several lows and notes identified throughout the codebase. To improve overall clarity, it is encouraged to add additional descriptions of the exact functionality of certain components, particularly the dispute router, to indicate clear trust assumptions such as the implementation of oracle fallback mechanisms.

The Across team is commended for their responsiveness and collaboration throughout the audit period.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to

the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system. These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.

Ready to secure your code?

[Request an Audit→](#)

Smart Contracts

Contracts Libraries

Contracts Wizard ↗

Contracts MCP ↗

Upgrades Plugin ↗

Open Source Tools

Relayer

Monitor

UI Builder ↗

Safe Utils ↗

Role Manager ↗

The Open Source Stack

Blockchains

Arbitrum

Base

Ethereum

Linea

Midnight

OP Mainnet

Polkadot

Starknet

Stellar

Zama

ZKsync

Services

Smart Contract Security Audit

Company

About Us

Why OpenZeppelin

News

Customer Stories

Careers

Brand Kit

Trust ↗

Terms & Policies

Terms of Service

Privacy Policy

Trademark Guidelines

AGPL License

Resources

Research

Blockchain Infrastructure Audit

ZKP Audit

Security Services

Solutions

Ecosystem Stack

Financial Institutions Stack

Documentation ↗

Stats

Forum ↗

Ethernaut CTF ↗



© 2026 Zeppelin Group Ltd